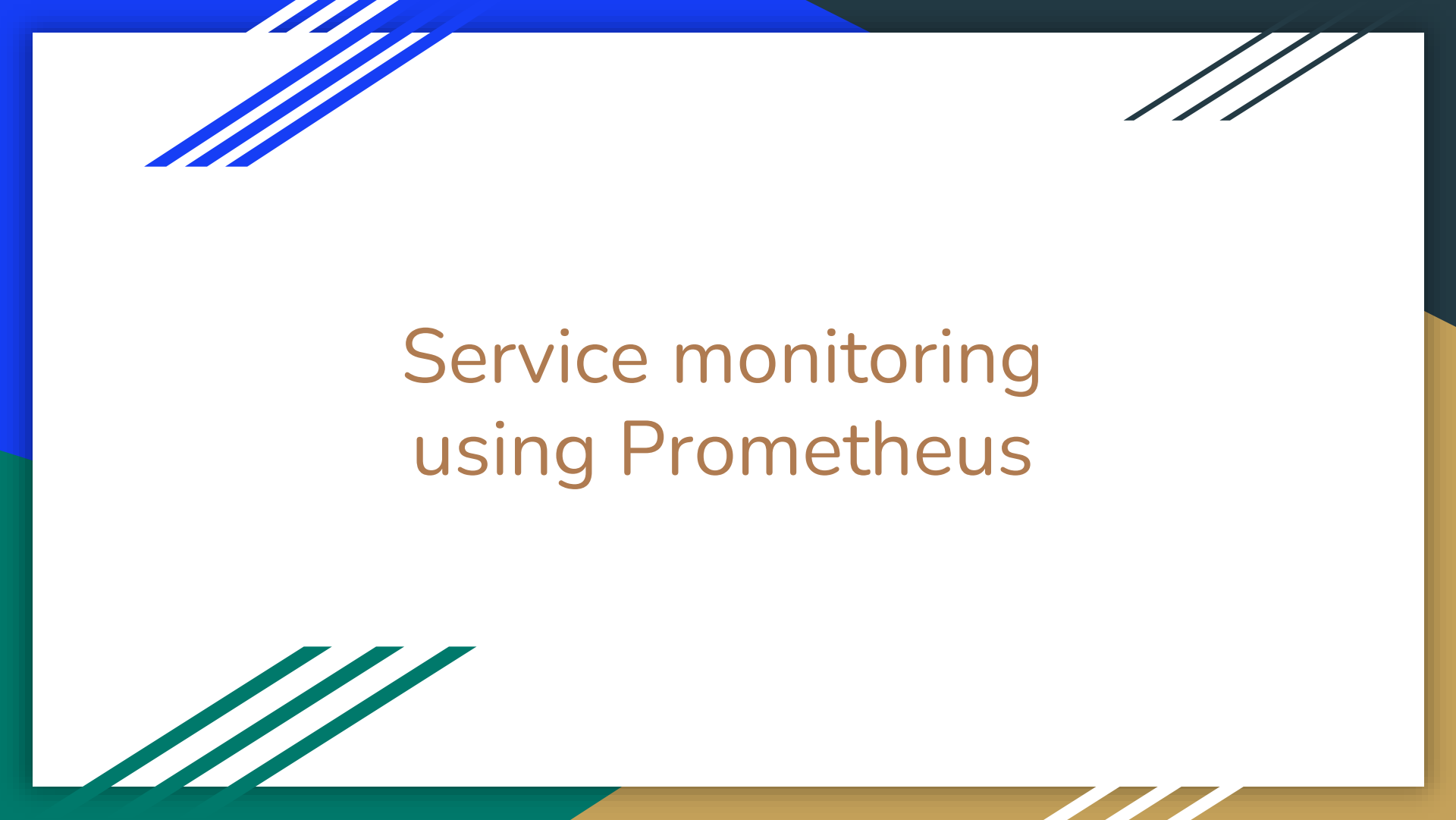


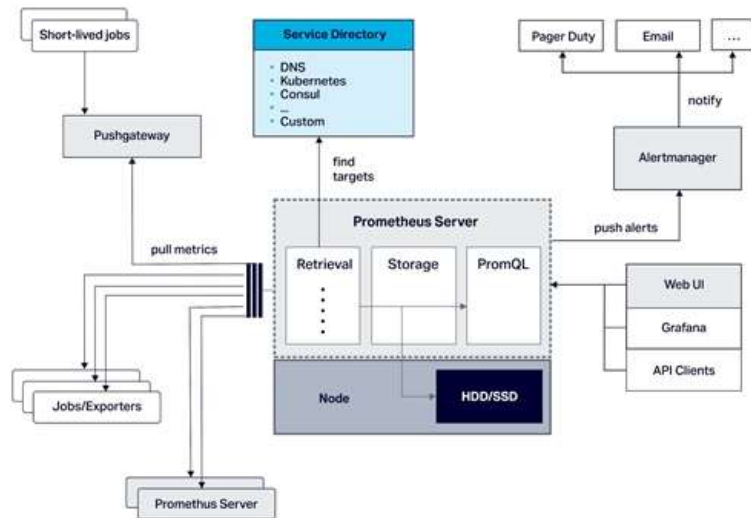


Service monitoring using Prometheus

Understanding the Prometheus Monitoring and Alert System

Originally developed by SoundCloud, then made open source and accepted as the second project in the Cloud Native Computing Foundation (CNCF) in 2016, Prometheus is an increasingly popular tool providing monitoring and alerts across applications and servers.

Prometheus's architecture looks like this:



Components

Prometheus' primary components include the Prometheus server (which handles service discovery, retrieval of metrics from monitored applications, storage of those metrics, and analysis of time-series data using the PromQL query language), a metrics data model, a simple built-in graphing GUI, and native Grafana support. Additional optional components include an alert manager (in which alerts can be defined in the query language) and a push gateway useful for monitoring short-lived applications.

Usually, tools for monitoring applications capture metrics via one of these three methods:

- **Instrumentation:** Adding custom code to the monitored application's source code.
- **Agents:** Adding special general-purpose code to the application environment designed to automatically capture standard metrics.
- **Spying:** Using interceptors or network taps to monitor calls or data flow between systems.

Components

Prometheus supports using a combination of instrumentation and agents (which it calls “exporters”). Instrumentation will need source code access, and makes it possible to capture custom metrics. It’s also programming language-agnostic, and has officially-supported client libraries available for Go, Java/Scala, Python, and Ruby. Many unofficial libraries are available as well (LISP, etc.), or you can write your own.

Many third-party exporters are available to enable automatic instrumentation of specific software, including popular databases, hardware, messaging systems, storage, HTTP, cloud APIs, logging, monitoring systems, and more. The available JMX exporter can export metrics for JVM-based applications, such as Kafka and Cassandra. At the same time, some software will expose metrics in the Prometheus format, rendering exporters unnecessary.

A node exporter is available for monitoring of host hardware and kernel metrics. The Prometheus Java client features collectors for garbage collection, memory pools, JMX, classloading, and thread counts, which can be added one at a time or registered all at once using `DefaultExports.initialize()`;

What Prometheus does well (and what it doesn't do at all)

Prometheus is great for monitoring metrics – and that's it. It's not an effective Application Performance Management (APM) tool, because it focuses solely on server-side metrics. It doesn't offer distributed call tracing, service topology discovery and visualization, performance analytics, or end user experience monitoring (although there is a github extension that can push user browser metrics to Prometheus). The flow of information with Prometheus is one-way, so it cannot be used for active control. Finally, Prometheus runs as a single server by default, but it can be scaled using a federation of servers.

Prometheus data modeling

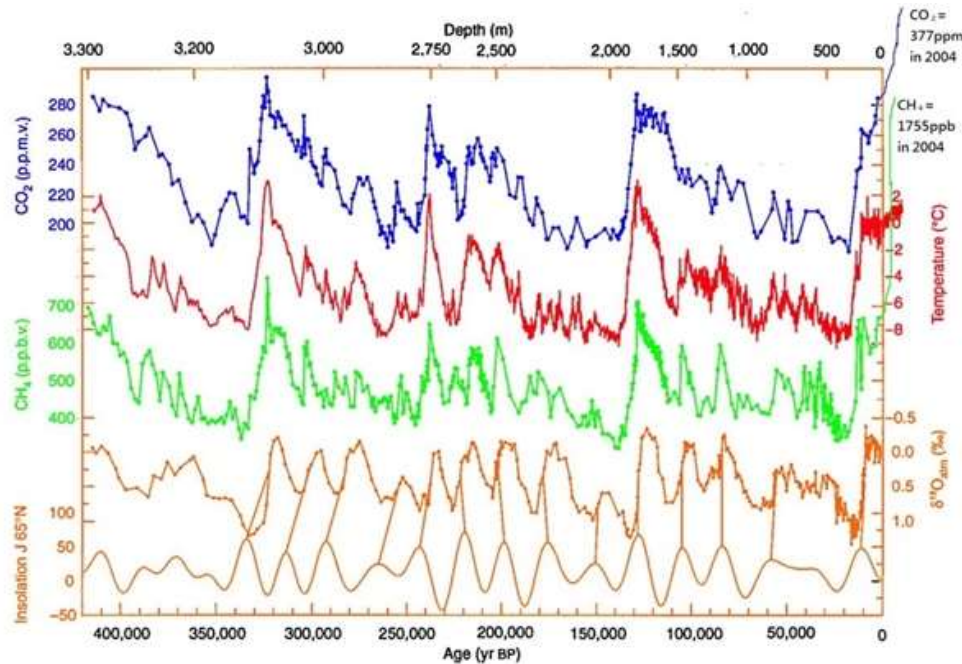


Fig 420,000 years of Antarctic ice core time-series data

Prometheus data modeling

Prometheus stores metrics as time-series data, such that metrics include streams of values (64-bit float) timestamped to the millisecond. Each metric has a name (a string), and uses a naming convention that includes the name of what is being monitored, the logical type, and the units of measure. Each metric also has a set of key:value pairs (the labeled dimensions). For example, a metric named `http_requests_total` could include labels for “method” (“GET”, “PUT”) and for “handler” (for instance: “/login”, “/search”). Prometheus also adds some labels to metrics automatically, including:

- job: The configured job name the target belongs to.
- Instance: The <host>:<port> portion of the URL scraped from the target.

Because units aren’t explicit, conversions from one unit to another can only be done manually in the query language (and should be performed extremely carefully).

Metric types

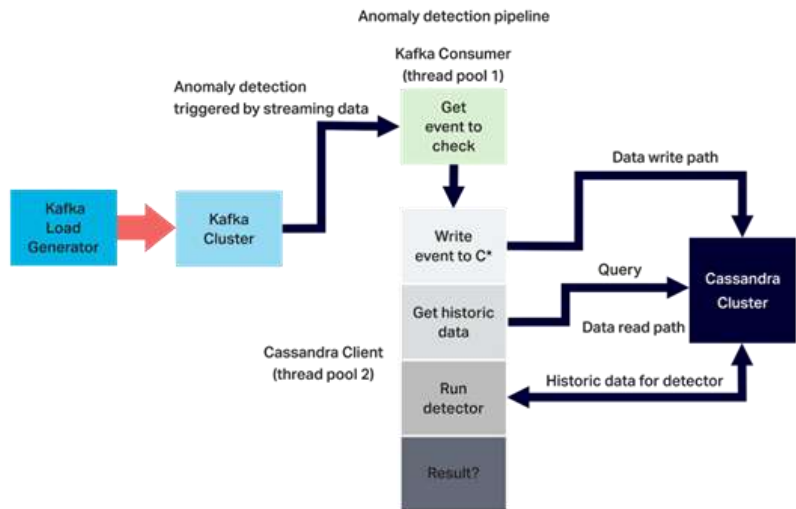
Prometheus offers these four different metric types:

1. Counter: A counter is useful for values that can only increase (the values can be reset to zero on restart).
2. Gauge: A gauge metric can be used for counts that go up and down, and also helpful for measured values that may rise and fall.
3. Histogram: A histogram samples observations, such as request durations or response sizes. It counts them in buckets that you can configure, while also offering the sum of all observed values.
4. Summary: Much like a histogram, a summary samples observations offers a total count of observations and the sum of observed values, while also calculating configurable quantiles over a sliding time window.

Putting Prometheus to Work for Anomalia Machina Monitoring

To understand what we recently wanted to monitor with the Anomalia Machina experiment, let's take a look at its functional architecture diagram:

Functional Architecture: Prototype



Putting Prometheus to Work for Anomalia Machina Monitoring

Using Prometheus, we looked to monitor “generic” application metrics, including the throughput (TPS) and response times of the Kafka load generator (Kafka producer), Kafka consumer, and Cassandra client (which detects anomalies). Additionally, we wanted to monitor some application-specific metrics, including the number of rows returned for each Cassandra read, and the number of anomalies detected. We also needed to monitor hardware metrics such as CPU for each of the AWS EC2 instances the application runs on, and to centralize monitoring by adding Kafka and Cassandra metrics there as well.

To accomplish this, we began by creating a simple test pipeline with three methods (producer, consumer, and detector). We then used a counter metric named “prometheusTest_requests_total” measured how many times each stage of the pipeline executes successfully, and a label called “stage” to tell the different stage counts apart (using “total” for the total pipeline count). We then used a second counter named “prometheusTest_anomalies_total” to count detected anomalies. And we used a gauge named “prometheusTest_duration_seconds” to record each stage’s duration in seconds (using a “stage” label to distinguish stages and a “total” label for the total pipeline duration).

Putting Prometheus to Work for Anomalia Machina Monitoring

The methods have instruments that increment the counter metrics each time a stage successfully executes or an anomaly is detected (using the `inc()` method), and set the time value of the gauge metric for each stage (using the `setToTime()` method).

Here's a code example:

```
import java.io.IOException;
import io.prometheus.client.Counter;
import io.prometheus.client.Gauge;
import io.prometheus.client.exporter.HTTPServer;
import io.prometheus.client.hotspot.DefaultExports;

// https://github.com/prometheus/client_java
// Demo of how we plan to use Prometheus Java client to instrument Anomalia Machina
// Note that the Anomalia Machina application will have Kafka Producer and Kafka Consumer
// So metrics from each will have different host/port combinations.
public class PrometheusBlog {
    static String appName = "prometheusTest";
    // counters can only increase in value (until process restart)
    // Execution count. Use a single Counter for all stages of the pipeline, stages
    static final Counter pipelineCounter = Counter.build()
        .name(appName + "_requests_total").help("Count of executions of pipeline stages")
        .labelNames("stage")
        .register();
    // In theory could also use pipelineCounter to count anomalies found using another
    // but less potential for confusion having another counter. Doesn't need a label
    static final Counter anomalyCounter = Counter.build()
        .name(appName + "_anomalies_total").help("Count of anomalies detected")
        .register();
    // A Gauge can go up and down, and is used to measure current value of some variable
    // pipelineGauge will measure duration in seconds of each stage using labels.
    static final Gauge pipelineGauge = Gauge.build()
        .name(appName + "_duration_seconds").help("Gauge of stage durations in seconds")
        .labelNames("stage")
        .register();

    public static void main(String[] args) {
        // Allow default JVM metrics to be exported
        DefaultExports.initialize();

        // Metrics are pulled by Prometheus, create an HTTP server as the endpoint
        // Note if there are multiple processes running on the same server need to change port
        // And add all IPs and port numbers to the Prometheus configuration file.
        HTTPServer server = null;
        try {
            server = new HTTPServer(1234);
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Putting Prometheus to Work for Anomalia Machina Monitoring

Here's a code example(Continue):

```
// now run 1000 executions of the complete pipeline with random time delays and
int max = 1000;
for (int i=0; i < max; i++)
{
    // total time for complete pipeline, and increment anomalyCounter
    pipelineGauge.labels("total").setToTime() -> {
        producer();
        consumer();
        if (detector())
            anomalyCounter.inc();
    };
    // total pipeline count
    pipelineCounter.labels("total").inc();
    System.out.println("i=" + i);

    // Increase the rate of execution
    try {
        Thread.sleep(max-i);
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
}
server.stop();
}

// the 3 stages of the pipeline, for each we increase the stage counter and set a
public static void producer() {
    class Local {}
    String name = local.class.getEnclosingMethod().getName();
    pipelineGauge.labels(name).setToTime() -> {
        try {
            Thread.sleep(1 + (long)(Math.random()*20));
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    };
    pipelineCounter.labels(name).inc();
}

public static void consumer() {
    class Local {}
    String name = local.class.getEnclosingMethod().getName();
    pipelineGauge.labels(name).setToTime() -> {
        try {
            Thread.sleep(1 + (long)(Math.random()*10));
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    };
}
```



```
pipelineCounter.labels(name).inc();
}

// detector returns true if anomaly detected else false
public static boolean detector() {
    class Local {}
    String name = local.class.getEnclosingMethod().getName();
    pipelineGauge.labels(name).setToTime() -> {
        try {
            Thread.sleep(1 + (long)(Math.random()*200));
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    };
    pipelineCounter.labels(name).inc();
    return (Math.random() > 0.95);
}
}
```

Putting Prometheus to Work for Anomalia Machina Monitoring

With sample code prepared, it's important to understand how to run Prometheus, and how Prometheus gets metric values from the code. In contrast to enterprise APM solutions that have metrics pushed to them, Prometheus receives metrics by polling ("scraping") instrumented code. This simply requires an HTTP server to run in the application code: the code above creates an HTTP server on port 1234 that enables Prometheus to scrape metrics.

Next, we needed to address Maven dependencies:



```
<!-- The client -->
<dependency>
  <groupId>io.prometheus</groupId>
  <artifactId>simpleclient</artifactId>
  <version>LATEST</version>
</dependency>
<!-- Hotspot JVM metrics-->
<dependency>
  <groupId>io.prometheus</groupId>
  <artifactId>simpleclient_hotspot</artifactId>
  <version>LATEST</version>
</dependency>
<!-- Exposition HTTPServer-->
<dependency>
  <groupId>io.prometheus</groupId>
  <artifactId>simpleclient_httpserver</artifactId>
  <version>LATEST</version>
</dependency>
<!-- Pushgateway exposition-->
<dependency>
  <groupId>io.prometheus</groupId>
  <artifactId>simpleclient_pushgateway</artifactId>
  <version>LATEST</version>
</dependency>
```

Putting Prometheus to Work for Anomalia Machina Monitoring

And lastly, we needed to tell Prometheus where do its scraping from. For simple deployments or testing purposes, it's adequate to add this information to the configuration file (the default is `prometheus.yml`):

```
global:
  scrape_interval: 15s # By default, scrape targets every 15 seconds.

# scrape_configs has jobs and targets to scrape for each.
scrape_configs:
  # job 1 is for testing prometheus instrumentation from multiple application processes.
  # The job name is added as a label job=<job_name> to any timeseries scraped from this config.
  - job_name: 'testprometheus'

    # Override the global default and scrape targets from this job every 5 seconds.
    scrape_interval: 5s

    # this is where to put multiple targets, e.g. for Kafka load generators and detectors
    static_configs:
      - targets: ['localhost:1234', 'localhost:1235']

  # job 2 provides operating system metrics (e.g. CPU, memory etc).
  - job_name: 'node'

    # Override the global default and scrape targets from this job every 5 seconds.
    scrape_interval: 5s

    static_configs:
      - targets: ['localhost:9100']
```

Putting Prometheus to Work for Anomalia Machina Monitoring

The configuration file also includes a job called “node,” using port 9100. This job provides node metrics – using it requires downloading the Prometheus node exporter and running it on the same server as the application. There are pros and cons to polling for metrics: polling too often can overload applications, while not polling often enough can produce troubling lags between when events happen and when they are detected.

At the same time, this system is robust and quite loosely coupled since applications are able to run without Prometheus, and Prometheus will simply continue attempting to poll unavailable applications until they become available. It’s also possible to poll a single application with multiple Prometheus servers. But if for any reason it isn’t possible to poll application metrics, or the application is highly transient, Prometheus features a push gateway that can be used as an alternative.

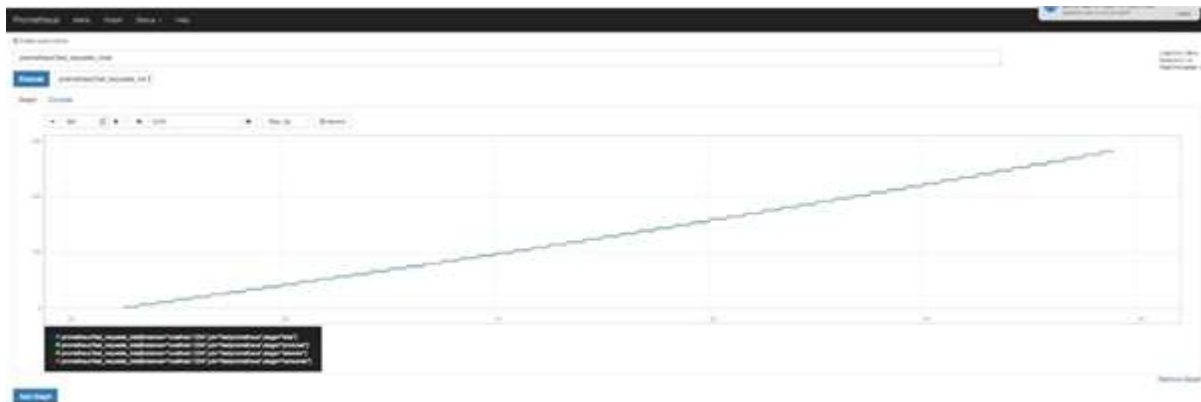
Initial Results Using Prometheus

Prometheus doesn't include default dashboards, so initially, we used expressions in our experiment. In the Prometheus interface, you'll find a scroll down menu from which you can select metric names (you can also do this via browser at <http://localhost:9090/metrics>). You can then enter these metrics in the expression box and execute them. Don't be discouraged if you encounter an error message and need to address some issue at this stage (this is a common experience).

Once the working expression is in place, you can view results in a table, or a graph if available for that result type. By default, expressions go back just five minutes to find data, and if data isn't available you'll receive an error. For the purpose of trying out Prometheus, you can leverage the fact that Prometheus monitors itself to explore the solution without needing an instrumented application to be available.

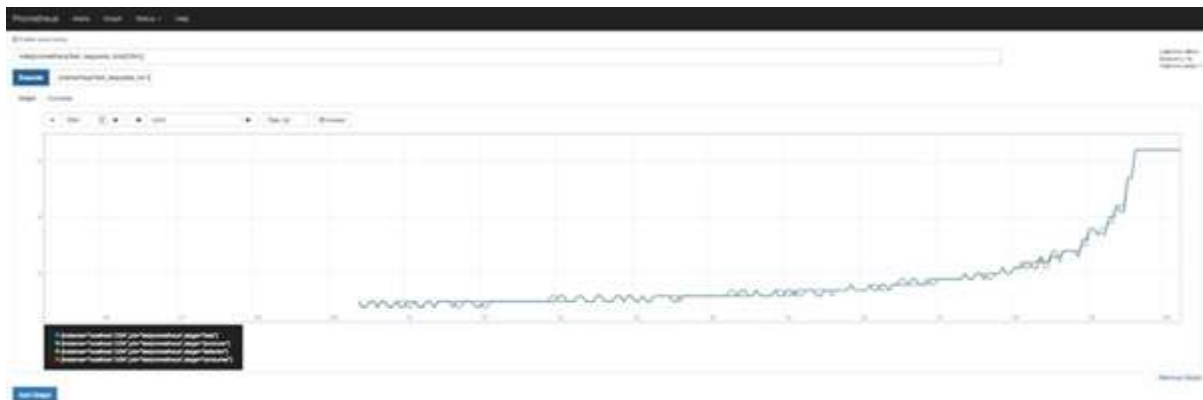
Graphing data

Because a counter metric simply increments in value, graphing a counter will just produce a line:



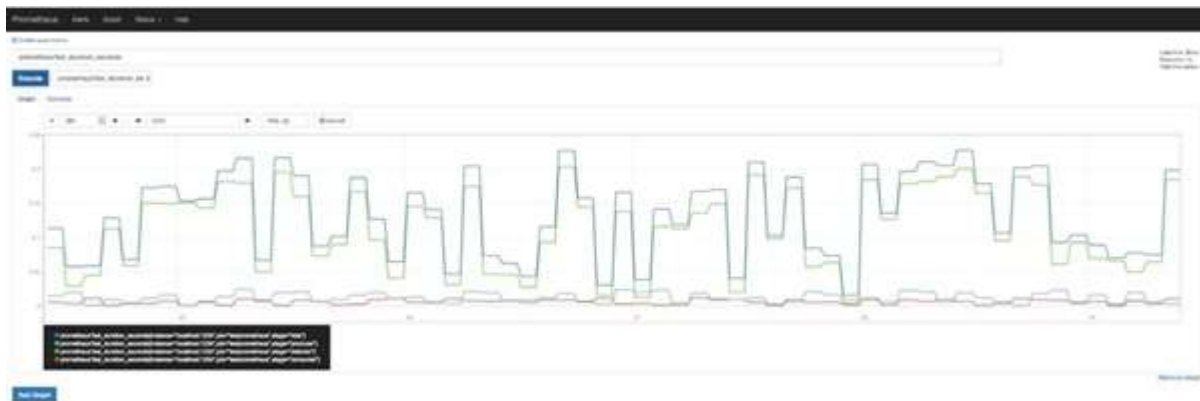
Graphing data

To get a rate graph based on a counter metric, use the **irate** or **rate** function.



Graphing data

In the example below, the graph displays pipeline stage durations, and doesn't require a rate function because it's a gauge instead of a counter:



Graphing data

While Prometheus' built-in graphing capabilities are limited – for example, you can't graph more than one metric on the same graph – adding Grafana offers vastly expanded capabilities. Grafana offers built-in Prometheus support, and is a must for serious graphing usage. Both the Prometheus and Grafana sites offer helpful documentation on using these tools together.

To use Grafana, install it and navigate your browser to <http://localhost:3000/>. Create a Prometheus data source, then add a Prometheus graph, using an expression as usual. For example, the graph below is able to display both duration and rate metrics:



Graphing data

One tip: if nothing is visible on the graph, the issue is probably that you aren't seeing the correct time range. A fast solution is to use a "Quick range" – the "Last 5 minutes" setting should be sufficient. You can also use rules to precompute rates, in order to potentially speed up aggregation.

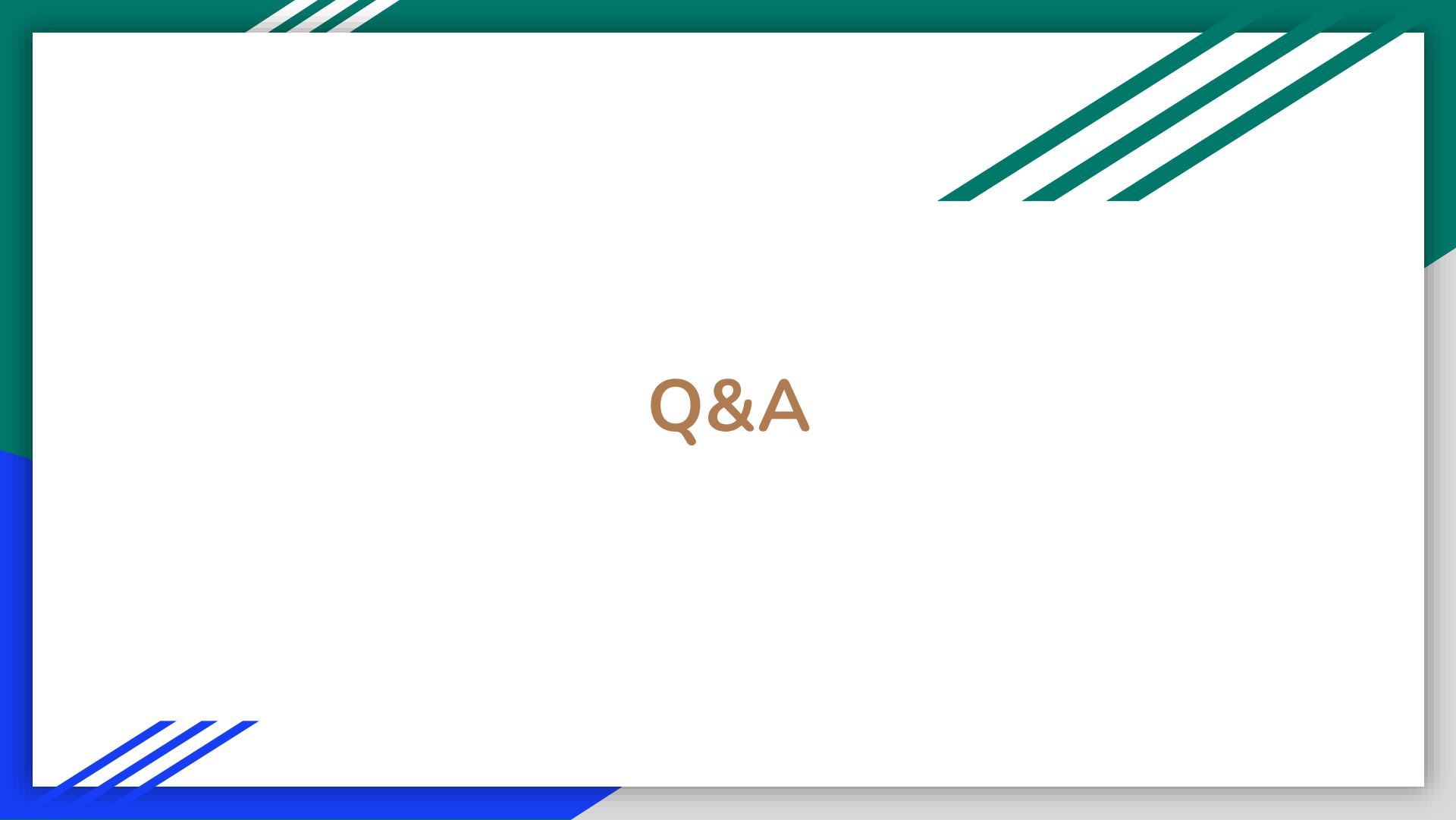
It's also possible to graph node metrics, such as CPU utilization. For example, you can compute CPU utilization as a percentage with this expression:

```
100 - (avg by (instance) (irate(node_cpu_seconds_total{job="node",mode="idle"}[5m])) * 100)
```

Summary

Finally, those using Cassandra with these tools may also find value in using Instaclustr's Cassandra Exporter for Prometheus, which is ideal for integrating Cassandra metrics from a self-managed cluster into your application monitoring leveraging Prometheus. The tool is well-documented, and provides a helpful starting point for developing a deeper understanding of Prometheus.

Having built our knowledge of how to use Prometheus to monitor an example application, we're able to monitor the actual Machine Anomalia application code and confidently put it into production.



Q&A